Métricas de performance en ML Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 El problema de predicción **3**
1.1 Concepto fundamental . . . 3
1.2 Flujo de decisión con predicción . . . 3

2 Clasificación binaria **4**
2.1 Problema de admisión académica . . . 4
2.2 Entrenar modelo de clasificación . . . 5

3 Thresholding (Umbralización) **6**
3.1 Concepto de probabilidades . . . 6
3.2 Efecto del umbral . . . 7
3.3 Visualizar distribución de probabilidades . . . 8

4 Matriz de confusión **9**
4.1 Concepto y estructura . . . 9
4.2 Calcular matriz de confusión . . . 10
4.3 Función personalizada para matriz de confusión . . . 11
4.4 Visualizar matriz de confusión . . . 11

5 Métricas de clasificación **13**
5.1 Resumen de métricas principales . . . 13

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

6 Accuracy (Exactitud) 14
6.1 Cálculo e interpretación . . . 14
6.2 Limitaciones del accuracy . . . 15

7 Precision (Precisión) 16
7.1 Cálculo e interpretación . . . 16
7.2 Ejemplo económico: Aprobación de créditos . . . 16

8 Recall (Sensibilidad) 18
8.1 Cálculo e interpretación . . . 18
8.2 Ejemplo económico: Detección de fraude . . . 18

9 F1-Score 20
9.1 Cálculo e interpretación . . . 20
9.2 Trade-off entre Precision y Recall . . . 20

10 Curva Precision-Recall 21
10.1 Generar curva Precision-Recall . . . 21
10.2 Visualizar curva Precision-Recall . . . 22

11 Curva ROC y AUC 23
11.1 Concepto de ROC . . . 23
11.2 Calcular TPR y FPR . . . 24
11.3 Visualizar curva ROC . . . 25
11.4 Interpretación del AUC . . . 26

12 Aplicaciones económicas 27
12.1 Aplicación 1: Predicción de incumplimiento crediticio . . . 27
12.2 Aplicación 2: Clasificación de empresas por riesgo de quiebra . . . 30

13 Ejercicios prácticos 33
13.1 Ejercicio 1: Optimizar umbral según costo . . . 33

14 Conclusión 36

15 Próximos pasos 37

16 Recursos adicionales 37

17 Publicaciones Similares 37

Métricas de performance en ML Python

En esta octava guía exploraremos el problema de predicción y las métricas fundamentales para evaluar modelos de clasificación. Estos conceptos son esenciales para evaluar modelos de machine learning aplicados a problemas económicos como predicción de incumplimiento crediticio, clasificación de empresas, predicción de pobreza, entre otros.

1 El problema de predicción

1.1 Concepto fundamental

En economía y ciencias sociales frecuentemente necesitamos predecir eventos futuros basándonos en información histórica:

- ¿Un cliente pagará o no pagará su crédito?
- ¿Una empresa quebrará o sobrevivirá?
- ¿Un hogar saldrá o permanecerá en pobreza?
- ¿Un estudiante completará o abandonará sus estudios?
- ¿Un producto se venderá o no se venderá?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import roc_curve, auc, precision_recall_curve
from sklearn.model_selection import train_test_split

# Configuración de gráficos
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (10, 6)
```

1.2 Flujo de decisión con predicción

```
print("""
FLUJO DE TOMA DE DECISIONES CON PREDICCIÓN

1. RECOLECCIÓN DE DATOS
     Variables explicativas (características)
     Variable objetivo (lo que queremos predecir)

2. ENTRENAMIENTO DEL MODELO
     Aprender patrones de los datos históricos
     Establecer relación entre X (predictores) y Y (objetivo)

3. PREDICCIÓN
     Aplicar modelo a nuevos casos
     Obtener probabilidades o clasificaciones
```

```
4. EVALUACIÓN
     Medir qué tan buenas son las predicciones
     Usar métricas apropiadas

5. DECISIÓN
     Usar predicción para tomar acción
     Ej: Aprobar/rechazar crédito, intervenir/no intervenir
""")
```

```
FLUJO DE TOMA DE DECISIONES CON PREDICCIÓN

1. RECOLECCIÓN DE DATOS
     Variables explicativas (características)
     Variable objetivo (lo que queremos predecir)

2. ENTRENAMIENTO DEL MODELO
     Aprender patrones de los datos históricos
     Establecer relación entre X (predictores) y Y (objetivo)

3. PREDICCIÓN
     Aplicar modelo a nuevos casos
     Obtener probabilidades o clasificaciones

4. EVALUACIÓN
     Medir qué tan buenas son las predicciones
     Usar métricas apropiadas

5. DECISIÓN
     Usar predicción para tomar acción
     Ej: Aprobar/rechazar crédito, intervenir/no intervenir
```

2 Clasificación binaria

2.1 Problema de admisión académica

```
# Crear dataset de ejemplo: admisión a programa de posgrado
np.random.seed(42)

# Datos simulados
n_estudiantes = 200

datos_admision = pd.DataFrame({
    'puntaje_examen': np.random.randint(500, 800, n_estudiantes),
    'gpa': np.random.uniform(2.0, 4.0, n_estudiantes),
    'experiencia_laboral': np.random.randint(0, 8, n_estudiantes),
    'publicaciones': np.random.randint(0, 5, n_estudiantes)
})
```

```
# Generar variable de admisión (1 = admitido, 0 = no admitido)
# Basado en una combinación de factores
score = (datos_admision['puntaje_examen'] - 500) * 0.002 + \
        datos_admision['gpa'] * 0.15 + \
        datos_admision['experiencia_laboral'] * 0.08 + \
        datos_admision['publicaciones'] * 0.10 + \
        np.random.normal(0, 0.2, n_estudiantes)

datos_admision['admitido'] = (score > 1.0).astype(int)

print("Dataset de admisiones:")
print(datos_admision.head(10))
print(f"\nDistribución de admisiones:")
print(datos_admision['admitido'].value_counts())
print(f"\nTasa de admisión: {datos_admision['admitido'].mean()*100:.1f}%")
```

```
Dataset de admisiones:
   puntaje_examen       gpa  experiencia_laboral  publicaciones  admitido
0             602  3.355129                    6              3         1
1             770  2.033176                    5              4         1
2             606  3.024186                    1              4         1
3             571  2.452992                    4              4         1
4             688  3.290346                    2              4         1
5             520  2.348733                    1              4         0
6             602  3.381875                    2              2         0
7             621  2.773471                    6              3         1
8             714  3.873460                    0              4         1
9             587  2.275042                    1              3         0

Distribución de admisiones:
admitido
1    140
0     60
Name: count, dtype: int64

Tasa de admisión: 70.0%
```

2.2 Entrenar modelo de clasificación

```
# Separar variables predictoras y objetivo
X = datos_admision[['puntaje_examen', 'gpa', 'experiencia_laboral', 'publicaciones']]
y = datos_admision['admitido']

# Dividir en conjunto de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
print(f"Tamaño conjunto entrenamiento: {len(X_train)}")
print(f"Tamaño conjunto prueba: {len(X_test)}")

# Entrenar modelo de regresión logística
modelo = LogisticRegression(random_state=42)
modelo.fit(X_train, y_train)

print("\nModelo entrenado exitosamente")
print(f"Coeficientes del modelo:")
for variable, coef in zip(X.columns, modelo.coef_[0]):
    print(f"  {variable}: {coef:.4f}")
```

```
Tamaño conjunto entrenamiento: 140
Tamaño conjunto prueba: 60

Modelo entrenado exitosamente
Coeficientes del modelo:
  puntaje_examen: 0.0089
  gpa: 0.7248
  experiencia_laboral: 0.5510
  publicaciones: 0.8955
```

3 Thresholding (Umbralización)

3.1 Concepto de probabilidades

```
# Obtener probabilidades de predicción
probabilidades = modelo.predict_proba(X_test)

print("Probabilidades de predicción (primeras 10 observaciones):")
print("\nÍndice | P(No admitido) | P(Admitido) | Predicción")
print("-" * 55)

for i in range(10):
    pred = "Admitido" if probabilidades[i, 1] > 0.5 else "No admitido"
    print(f"{i:6d} | {probabilidades[i, 0]:14.4f} | {probabilidades[i, 1]:11.4f} | {p

# Crear DataFrame con resultados
resultados = pd.DataFrame({
    'prob_no_admitido': probabilidades[:, 0],
    'prob_admitido': probabilidades[:, 1],
    'real': y_test.values
})

print("\n\nEstadísticas de probabilidades:")
print(resultados['prob_admitido'].describe())
```

```
Probabilidades de predicción (primeras 10 observaciones):

Índice | P(No admitido) | P(Admitido) | Predicción
-----------------------------------------------------
     0 |         0.2196 |      0.7804 | Admitido
     1 |         0.1008 |      0.8992 | Admitido
     2 |         0.4035 |      0.5965 | Admitido
     3 |         0.2008 |      0.7992 | Admitido
     4 |         0.0295 |      0.9705 | Admitido
     5 |         0.7350 |      0.2650 | No admitido
     6 |         0.0866 |      0.9134 | Admitido
     7 |         0.5659 |      0.4341 | No admitido
     8 |         0.0246 |      0.9754 | Admitido
     9 |         0.1647 |      0.8353 | Admitido


Estadísticas de probabilidades:
count    60.000000
mean      0.685291
std       0.275099
min       0.078084
25%       0.498738
50%       0.780514
75%       0.928637
max       0.998200
Name: prob_admitido, dtype: float64
```

3.2 Efecto del umbral

```python
# Experimentar con diferentes umbrales
umbrales = [0.3, 0.5, 0.7, 0.9]

print("EFECTO DEL UMBRAL EN LA CLASIFICACIÓN")
print("=" * 70)

for umbral in umbrales:
    # Aplicar umbral
    predicciones = (probabilidades[:, 1] > umbral).astype(int)

    # Contar predicciones
    n_positivos = predicciones.sum()
    tasa_positivos = (n_positivos / len(predicciones)) * 100

    print(f"\nUmbral: {umbral}")
    print(f"  Predicciones positivas: {n_positivos} ({tasa_positivos:.1f}%)")
    print(f"  Predicciones negativas: {len(predicciones) - n_positivos}")
```

```
EFECTO DEL UMBRAL EN LA CLASIFICACIÓN
```

```
======================================================================

Umbral: 0.3
  Predicciones positivas: 53 (88.3%)
  Predicciones negativas: 7

Umbral: 0.5
  Predicciones positivas: 45 (75.0%)
  Predicciones negativas: 15

Umbral: 0.7
  Predicciones positivas: 34 (56.7%)
  Predicciones negativas: 26

Umbral: 0.9
  Predicciones positivas: 18 (30.0%)
  Predicciones negativas: 42
```

3.3 Visualizar distribución de probabilidades

```python
# Crear gráfico de distribución de probabilidades
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

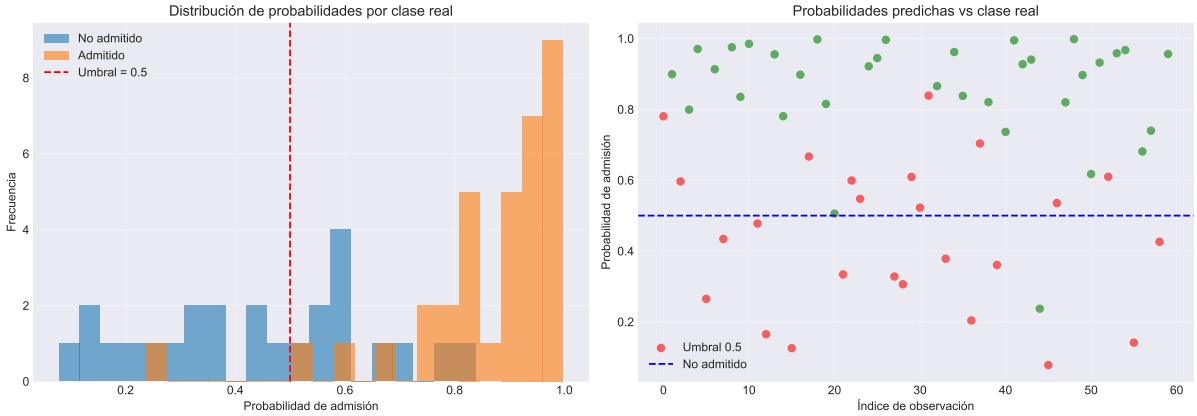
# Histograma por clase real
for clase, nombre in [(0, 'No admitido'), (1, 'Admitido')]:
    mask = resultados['real'] == clase
    axes[0].hist(resultados.loc[mask, 'prob_admitido'],
                 bins=20, alpha=0.6, label=nombre)

axes[0].axvline(x=0.5, color='red', linestyle='--', label='Umbral = 0.5')
axes[0].set_xlabel('Probabilidad de admisión')
axes[0].set_ylabel('Frecuencia')
axes[0].set_title('Distribución de probabilidades por clase real')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Scatter plot
colores = ['red' if x == 0 else 'green' for x in resultados['real']]
axes[1].scatter(range(len(resultados)),
                resultados['prob_admitido'],
                c=colores, alpha=0.6)
axes[1].axhline(y=0.5, color='blue', linestyle='--', label='Umbral = 0.5')
axes[1].set_xlabel('Índice de observación')
axes[1].set_ylabel('Probabilidad de admisión')
axes[1].set_title('Probabilidades predichas vs clase real')
axes[1].legend(['Umbral 0.5', 'No admitido', 'Admitido'])
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('distribucion_probabilidades.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'distribucion_probabilidades.png'")
```

```
Gráfico guardado como 'distribucion_probabilidades.png'
```



4 Matriz de confusión

4.1 Concepto y estructura

La matriz de confusión es una tabla que describe el desempeño de un modelo de clasificación.

```
print("""
MATRIZ DE CONFUSIÓN - ESTRUCTURA

                    Predicción
                 Negativo  Positivo

Real  Negativo      TN         FP

      Positivo      FN         TP


Donde:
- TN (True Negative): Negativos correctamente identificados
- FP (False Positive): Negativos incorrectamente clasificados como positivos
- FN (False Negative): Positivos incorrectamente clasificados como negativos
- TP (True Positive): Positivos correctamente identificados
""")
```

```
MATRIZ DE CONFUSIÓN - ESTRUCTURA

                    Predicción
                 Negativo  Positivo
```

```
Real  Negativo      TN         FP

      Positivo      FN         TP
```

```
Donde:
- TN (True Negative): Negativos correctamente identificados
- FP (False Positive): Negativos incorrectamente clasificados como positivos
- FN (False Negative): Positivos incorrectamente clasificados como negativos
- TP (True Positive): Positivos correctamente identificados
```

4.2 Calcular matriz de confusión

```python
# Predicciones con umbral 0.5
y_pred = modelo.predict(X_test)

# Calcular matriz de confusión
cm = confusion_matrix(y_test, y_pred)

print("MATRIZ DE CONFUSIÓN")
print("=" * 50)
print(f"\n                  Predicción")
print(f"               No admitido  Admitido")
print(f"Real No admitido    {cm[0,0]:4d}       {cm[0,1]:4d}")
print(f"     Admitido       {cm[1,0]:4d}       {cm[1,1]:4d}")

# Extraer componentes
TN, FP, FN, TP = cm.ravel()

print(f"\n\nComponentes de la matriz:")
print(f"  Verdaderos Negativos (TN): {TN}")
print(f"  Falsos Positivos (FP): {FP}")
print(f"  Falsos Negativos (FN): {FN}")
print(f"  Verdaderos Positivos (TP): {TP}")
```

```
MATRIZ DE CONFUSIÓN
==================================================

                Predicción
              No admitido  Admitido
Real No admitido      14         11
     Admitido          1         34


Componentes de la matriz:
  Verdaderos Negativos (TN): 14
  Falsos Positivos (FP): 11
  Falsos Negativos (FN): 1
  Verdaderos Positivos (TP): 34
```

4.3 Función personalizada para matriz de confusión

```
def calcular_matriz_confusion(y_real, y_pred):
    """
    Calcula matriz de confusión y sus componentes
    """
    # Convertir a arrays booleanos
    y_real = np.array(y_real).astype(bool)
    y_pred = np.array(y_pred).astype(bool)

    # Calcular componentes
    TP = (y_real & y_pred).sum()
    TN = (~y_real & ~y_pred).sum()
    FP = (~y_real & y_pred).sum()
    FN = (y_real & ~y_pred).sum()

    # Crear matriz
    matriz = np.array([[TN, FP],
                       [FN, TP]])

    return matriz, TP, TN, FP, FN

# Usar función personalizada
matriz, TP, TN, FP, FN = calcular_matriz_confusion(y_test, y_pred)

print("Matriz de confusión (función personalizada):")
print(matriz)
print(f"\nVerificación: TP={TP}, TN={TN}, FP={FP}, FN={FN}")
```

```
Matriz de confusión (función personalizada):
[[14 11]
 [ 1 34]]

Verificación: TP=34, TN=14, FP=11, FN=1
```

4.4 Visualizar matriz de confusión

```
def graficar_matriz_confusion(cm, nombres_clases=['No', 'Sí']):
    """
    Visualiza la matriz de confusión
    """
    fig, ax = plt.subplots(figsize=(8, 6))

    im = ax.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    ax.figure.colorbar(im, ax=ax)

    ax.set(xticks=np.arange(cm.shape[1]),
```

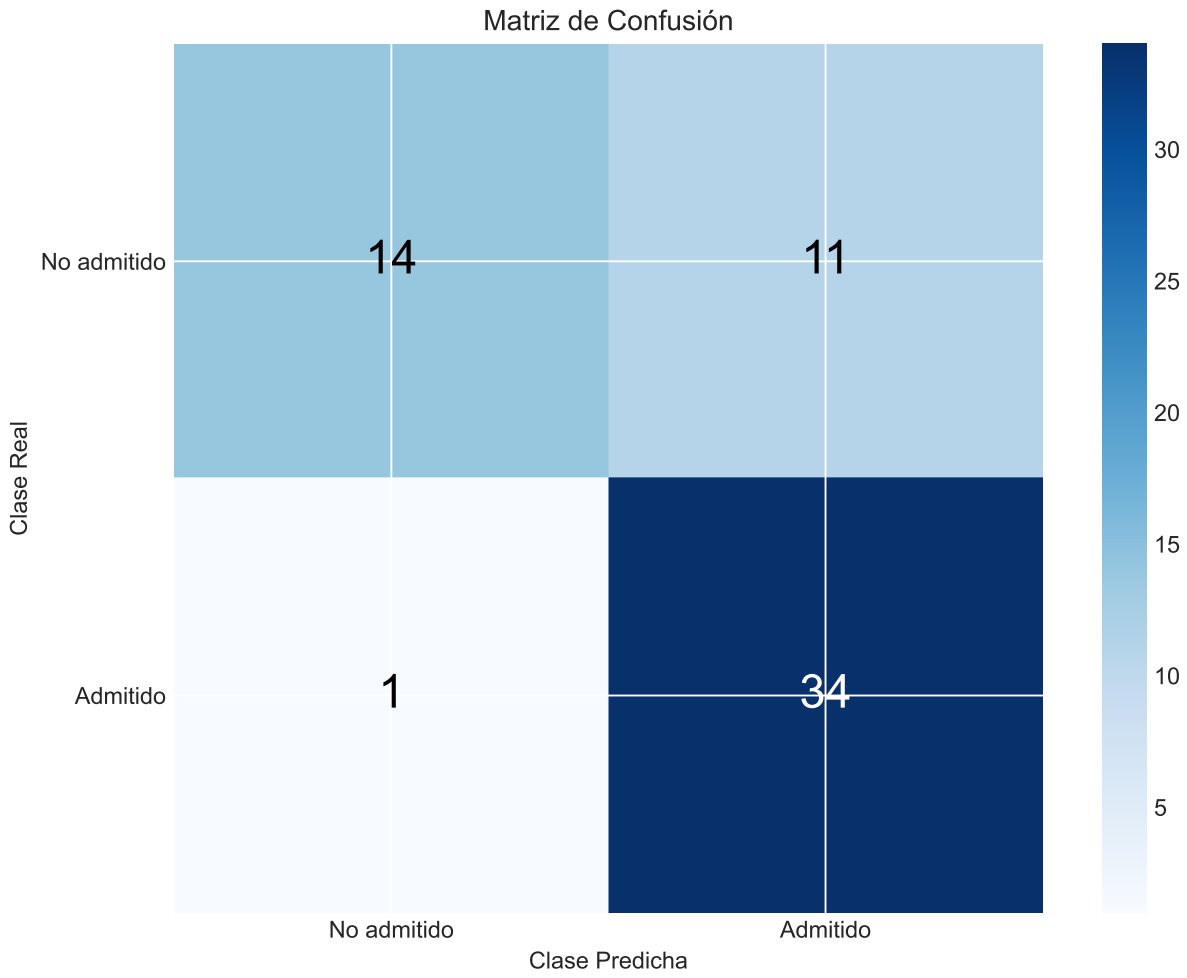
```
            yticks=np.arange(cm.shape[0]),
            xticklabels=nombres_clases,
            yticklabels=nombres_clases,
            ylabel='Clase Real',
            xlabel='Clase Predicha',
            title='Matriz de Confusión')

    # Anotar celdas con valores
    fmt = 'd'
    thresh = cm.max() / 2.
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, format(cm[i, j], fmt),
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black",
                    fontsize=20)

    plt.tight_layout()
    return fig

# Graficar
fig = graficar_matriz_confusion(cm, nombres_clases=['No admitido', 'Admitido'])
plt.savefig('matriz_confusion.png', dpi=300, bbox_inches='tight')
print("Matriz de confusión guardada como 'matriz_confusion.png'")
```

```
Matriz de confusión guardada como 'matriz_confusion.png'
```



5 Métricas de clasificación

5.1 Resumen de métricas principales

```
print("""
MÉTRICAS DE CLASIFICACIÓN

1. ACCURACY (Exactitud)
   - Proporción total de predicciones correctas
   - Fórmula: (TP + TN) / (TP + TN + FP + FN)
   - Cuándo usar: Clases balanceadas

2. PRECISION (Precisión)
   - De las predicciones positivas, cuántas son correctas
   - Fórmula: TP / (TP + FP)
   - Cuándo usar: Costo alto de falsos positivos
   - Ejemplo: Diagnóstico médico (evitar alarmas falsas)

3. RECALL (Sensibilidad/Exhaustividad)
   - De los casos positivos reales, cuántos detectamos
   - Fórmula: TP / (TP + FN)
```

```
   - Cuándo usar: Costo alto de falsos negativos
   - Ejemplo: Detección de fraude (no perder casos reales)

4. F1-SCORE
   - Media armónica de precision y recall
   - Fórmula: 2 * (Precision * Recall) / (Precision + Recall)
   - Cuándo usar: Balance entre precision y recall

5. SPECIFICITY (Especificidad)
   - De los casos negativos reales, cuántos detectamos
   - Fórmula: TN / (TN + FP)
""")
```

```
MÉTRICAS DE CLASIFICACIÓN

1. ACCURACY (Exactitud)
   - Proporción total de predicciones correctas
   - Fórmula: (TP + TN) / (TP + TN + FP + FN)
   - Cuándo usar: Clases balanceadas

2. PRECISION (Precisión)
   - De las predicciones positivas, cuántas son correctas
   - Fórmula: TP / (TP + FP)
   - Cuándo usar: Costo alto de falsos positivos
   - Ejemplo: Diagnóstico médico (evitar alarmas falsas)

3. RECALL (Sensibilidad/Exhaustividad)
   - De los casos positivos reales, cuántos detectamos
   - Fórmula: TP / (TP + FN)
   - Cuándo usar: Costo alto de falsos negativos
   - Ejemplo: Detección de fraude (no perder casos reales)

4. F1-SCORE
   - Media armónica de precision y recall
   - Fórmula: 2 * (Precision * Recall) / (Precision + Recall)
   - Cuándo usar: Balance entre precision y recall

5. SPECIFICITY (Especificidad)
   - De los casos negativos reales, cuántos detectamos
   - Fórmula: TN / (TN + FP)
```

6 Accuracy (Exactitud)

6.1 Cálculo e interpretación

```python
def calcular_accuracy(y_real, y_pred):
    """Calcula accuracy"""
    _, TP, TN, FP, FN = calcular_matriz_confusion(y_real, y_pred)
```

```
    accuracy = (TP + TN) / (TP + TN + FP + FN)
    return accuracy

# Calcular accuracy
accuracy = calcular_accuracy(y_test, y_pred)
accuracy_sklearn = modelo.score(X_test, y_test)

print(f"Accuracy (función propia): {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Accuracy (sklearn): {accuracy_sklearn:.4f} ({accuracy_sklearn*100:.2f}%)")

print(f"\nInterpretación:")
print(f"El modelo clasifica correctamente el {accuracy*100:.1f}% de los casos")
```

```
Accuracy (función propia): 0.8000 (80.00%)
Accuracy (sklearn): 0.8000 (80.00%)

Interpretación:
El modelo clasifica correctamente el 80.0% de los casos
```

6.2 Limitaciones del accuracy

```
# Ejemplo con dataset desbalanceado
print("\nEJEMPLO: LIMITACIÓN DEL ACCURACY CON CLASES DESBALANCEADAS")
print("=" * 70)

# Dataset muy desbalanceado (95% negativos, 5% positivos)
n = 1000
y_desbalanceado = np.array([0]*950 + [1]*50)

# Modelo que siempre predice negativo
y_pred_siempre_negativo = np.zeros(n)

acc = calcular_accuracy(y_desbalanceado, y_pred_siempre_negativo)
print(f"\nModelo que predice siempre 'No':")
print(f"Accuracy: {acc:.4f} ({acc*100:.1f}%)")
print("\n¡Accuracy alto pero modelo inútil!")
print("No detecta ningún caso positivo.")

# Matriz de confusión
cm_desbalanceado = confusion_matrix(y_desbalanceado, y_pred_siempre_negativo)
print(f"\nMatriz de confusión:")
print(cm_desbalanceado)
print("\nObservación: TP = 0 (no detecta ningún positivo)")
```

```
EJEMPLO: LIMITACIÓN DEL ACCURACY CON CLASES DESBALANCEADAS
======================================================================
```

```
Modelo que predice siempre 'No':
Accuracy: 0.9500 (95.0%)

¡Accuracy alto pero modelo inútil!
No detecta ningún caso positivo.

Matriz de confusión:
[[950   0]
 [ 50   0]]

Observación: TP = 0 (no detecta ningún positivo)
```

7 Precision (Precisión)

7.1 Cálculo e interpretación

```python
def calcular_precision(y_real, y_pred):
    """Calcula precision"""
    _, TP, TN, FP, FN = calcular_matriz_confusion(y_real, y_pred)

    if TP + FP == 0:
        return 0.0

    precision = TP / (TP + FP)
    return precision

# Calcular precision
precision = calcular_precision(y_test, y_pred)

print(f"Precision: {precision:.4f} ({precision*100:.2f}%)")
print(f"\nInterpretación:")
print(f"De los estudiantes que el modelo predice como 'admitidos',")
print(f"el {precision*100:.1f}% realmente son admitidos.")
print(f"\nFalsos positivos: {FP} estudiantes")
print("(Predichos como admitidos pero en realidad no lo fueron)")
```

```
Precision: 0.7556 (75.56%)

Interpretación:
De los estudiantes que el modelo predice como 'admitidos',
el 75.6% realmente son admitidos.

Falsos positivos: 11 estudiantes
(Predichos como admitidos pero en realidad no lo fueron)
```

7.2 Ejemplo económico: Aprobación de créditos

```python
print("\n\nEJEMPLO ECONÓMICO: APROBACIÓN DE CRÉDITOS")
print("=" * 70)

# Simular decisiones de crédito
n_creditos = 1000
y_real_pagara = np.random.choice([0, 1], n_creditos, p=[0.2, 0.8])  # 80% pagan
y_pred_aprobar = np.random.choice([0, 1], n_creditos, p=[0.15, 0.85])  # Modelo aprue

# Calcular métricas
_, TP, TN, FP, FN = calcular_matriz_confusion(y_real_pagara, y_pred_aprobar)
precision_credito = calcular_precision(y_real_pagara, y_pred_aprobar)

print(f"\nResultados de {n_creditos} solicitudes:")
print(f"  Créditos aprobados por modelo: {TP + FP}")
print(f"  De estos, pagarán: {TP} ({precision_credito*100:.1f}%)")
print(f"  De estos, NO pagarán: {FP} (Pérdida por falsos positivos)")

# Calcular pérdida
monto_promedio = 10000
perdida_por_fp = FP * monto_promedio

print(f"\n\nAnálisis financiero:")
print(f"  Monto promedio por crédito: S/ {monto_promedio:,}")
print(f"  Falsos positivos: {FP}")
print(f"  Pérdida estimada: S/ {perdida_por_fp:,}")
print(f"\nConlusión: Alta precision reduce pérdidas por morosidad")
```

```
EJEMPLO ECONÓMICO: APROBACIÓN DE CRÉDITOS
======================================================================

Resultados de 1000 solicitudes:
  Créditos aprobados por modelo: 847
  De estos, pagarán: 680 (80.3%)
  De estos, NO pagarán: 167 (Pérdida por falsos positivos)


Análisis financiero:
  Monto promedio por crédito: S/ 10,000
  Falsos positivos: 167
  Pérdida estimada: S/ 1,670,000

Conlusión: Alta precision reduce pérdidas por morosidad
```

8 Recall (Sensibilidad)

8.1 Cálculo e interpretación

```
def calcular_recall(y_real, y_pred):
    """Calcula recall (sensibilidad)"""
    _, TP, TN, FP, FN = calcular_matriz_confusion(y_real, y_pred)

    if TP + FN == 0:
        return 0.0

    recall = TP / (TP + FN)
    return recall

# Calcular recall
recall = calcular_recall(y_test, y_pred)

print(f"Recall: {recall:.4f} ({recall*100:.2f}%)")
print(f"\nInterpretación:")
print(f"De los estudiantes que realmente fueron admitidos,")
print(f"el modelo detectó correctamente al {recall*100:.1f}%.")
print(f"\nFalsos negativos: {FN} estudiantes")
print("(No fueron identificados como admitidos cuando sí lo fueron)")
```

```
Recall: 0.9714 (97.14%)

Interpretación:
De los estudiantes que realmente fueron admitidos,
el modelo detectó correctamente al 97.1%.

Falsos negativos: 118 estudiantes
(No fueron identificados como admitidos cuando sí lo fueron)
```

8.2 Ejemplo económico: Detección de fraude

```
print("\n\nEJEMPLO ECONÓMICO: DETECCIÓN DE FRAUDE")
print("=" * 70)

# Simular transacciones
n_trans = 10000
y_real_fraude = np.random.choice([0, 1], n_trans, p=[0.98, 0.02])  # 2% fraude
y_pred_fraude = np.random.choice([0, 1], n_trans, p=[0.97, 0.03])  # Modelo detecta 3

# Calcular métricas
_, TP, TN, FP, FN = calcular_matriz_confusion(y_real_fraude, y_pred_fraude)
recall_fraude = calcular_recall(y_real_fraude, y_pred_fraude)
precision_fraude = calcular_precision(y_real_fraude, y_pred_fraude)
```

```python
print(f"\nResultados de {n_trans:,} transacciones:")
print(f"  Fraudes reales: {TP + FN}")
print(f"  Fraudes detectados: {TP} ({recall_fraude*100:.1f}% de los reales)")
print(f"  Fraudes no detectados: {FN} ¡CRÍTICO!")

# Calcular pérdida
monto_promedio_fraude = 5000
perdida_por_fn = FN * monto_promedio_fraude

print(f"\n\nAnálisis financiero:")
print(f"  Monto promedio por fraude: S/ {monto_promedio_fraude:,}")
print(f"  Fraudes no detectados (FN): {FN}")
print(f"  Pérdida por fraudes no detectados: S/ {perdida_por_fn:,}")

# Costo de investigar falsos positivos
costo_investigacion = 100
costo_fp = FP * costo_investigacion
print(f"\n  Alertas falsas (FP): {FP}")
print(f"  Costo de investigar falsas alarmas: S/ {costo_fp:,}")

print(f"\n\nConclusión: En detección de fraude, es MÁS IMPORTANTE el Recall")
print("(No queremos perder fraudes reales, aunque tengamos falsas alarmas)")
```

```
EJEMPLO ECONÓMICO: DETECCIÓN DE FRAUDE
======================================================================

Resultados de 10,000 transacciones:
  Fraudes reales: 198
  Fraudes detectados: 4 (2.0% de los reales)
  Fraudes no detectados: 194 ¡CRÍTICO!


Análisis financiero:
  Monto promedio por fraude: S/ 5,000
  Fraudes no detectados (FN): 194
  Pérdida por fraudes no detectados: S/ 970,000

  Alertas falsas (FP): 301
  Costo de investigar falsas alarmas: S/ 30,100


Conclusión: En detección de fraude, es MÁS IMPORTANTE el Recall
(No queremos perder fraudes reales, aunque tengamos falsas alarmas)
```

9 F1-Score

9.1 Cálculo e interpretación

```
def calcular_f1(y_real, y_pred):
    """Calcula F1-score"""
    precision = calcular_precision(y_real, y_pred)
    recall = calcular_recall(y_real, y_pred)

    if precision + recall == 0:
        return 0.0

    f1 = 2 * (precision * recall) / (precision + recall)
    return f1

# Calcular F1
f1 = calcular_f1(y_test, y_pred)

print(f"Métricas del modelo:")
print(f"  Precision: {precision:.4f}")
print(f"  Recall: {recall:.4f}")
print(f"  F1-Score: {f1:.4f}")

print(f"\n\nInterpretación del F1-Score:")
print("El F1-Score es la media armónica de Precision y Recall.")
print(f"Un F1 de {f1:.2f} indica un {'buen' if f1 > 0.7 else 'regular' if f1 > 0.5 el
```

```
Métricas del modelo:
  Precision: 0.7556
  Recall: 0.9714
  F1-Score: 0.8500


Interpretación del F1-Score:
El F1-Score es la media armónica de Precision y Recall.
Un F1 de 0.85 indica un buen balance entre ambas métricas.
```

9.2 Trade-off entre Precision y Recall

```
# Demostrar trade-off
umbrales = [0.1, 0.3, 0.5, 0.7, 0.9]

print("\n\nTRADE-OFF: PRECISION VS RECALL")
print("=" * 70)
print(f"{'Umbral':<10} {'Precision':<12} {'Recall':<12} {'F1-Score':<12}")
print("-" * 70)
```

```
for umbral in umbrales:
    y_pred_umbral = (probabilidades[:, 1] > umbral).astype(int)

    prec = calcular_precision(y_test, y_pred_umbral)
    rec = calcular_recall(y_test, y_pred_umbral)
    f1_temp = calcular_f1(y_test, y_pred_umbral)

    print(f"{umbral:<10.1f} {prec:<12.4f} {rec:<12.4f} {f1_temp:<12.4f}")

print("\nObservación:")
print("- Umbral bajo → Mayor Recall (detecta más positivos) pero menor Precision")
print("- Umbral alto → Mayor Precision (menos falsas alarmas) pero menor Recall")
```

```
TRADE-OFF: PRECISION VS RECALL
======================================================================
Umbral     Precision    Recall       F1-Score
----------------------------------------------------------------------
0.1        0.5932       1.0000       0.7447
0.3        0.6415       0.9714       0.7727
0.5        0.7556       0.9714       0.8500
0.7        0.9118       0.8857       0.8986
0.9        1.0000       0.5143       0.6792

Observación:
- Umbral bajo → Mayor Recall (detecta más positivos) pero menor Precision
- Umbral alto → Mayor Precision (menos falsas alarmas) pero menor Recall
```

10 Curva Precision-Recall

10.1 Generar curva Precision-Recall

```
def calcular_curva_precision_recall(y_real, probabilidades):
    """
    Calcula curva Precision-Recall para diferentes umbrales
    """
    # Obtener umbrales únicos
    umbrales = np.sort(np.unique(probabilidades))

    precision_list = []
    recall_list = []

    for umbral in umbrales:
        y_pred = (probabilidades >= umbral).astype(int)

        prec = calcular_precision(y_real, y_pred)
        rec = calcular_recall(y_real, y_pred)
```

```
        precision_list.append(prec)
        recall_list.append(rec)

    return np.array(recall_list), np.array(precision_list), umbrales

# Calcular curva
recall_curva, precision_curva, umbrales_pr = calcular_curva_precision_recall(
    y_test, probabilidades[:, 1]
)

# También usar sklearn
precision_sklearn, recall_sklearn, _ = precision_recall_curve(
    y_test, probabilidades[:, 1]
)
```

10.2 Visualizar curva Precision-Recall

```
# Crear gráfico
fig, ax = plt.subplots(figsize=(10, 8))

# Curva propia
ax.plot(recall_curva, precision_curva, 'b-', linewidth=2,
        label='Curva PR (implementación propia)')

# Curva sklearn
ax.plot(recall_sklearn, precision_sklearn, 'r--', linewidth=2, alpha=0.7,
        label='Curva PR (sklearn)')

# Línea de referencia (clasificador aleatorio)
proporcion_positivos = y_test.mean()
ax.axhline(y=proporcion_positivos, color='gray', linestyle='--',
           label=f'Clasificador aleatorio ({proporcion_positivos:.2f})')

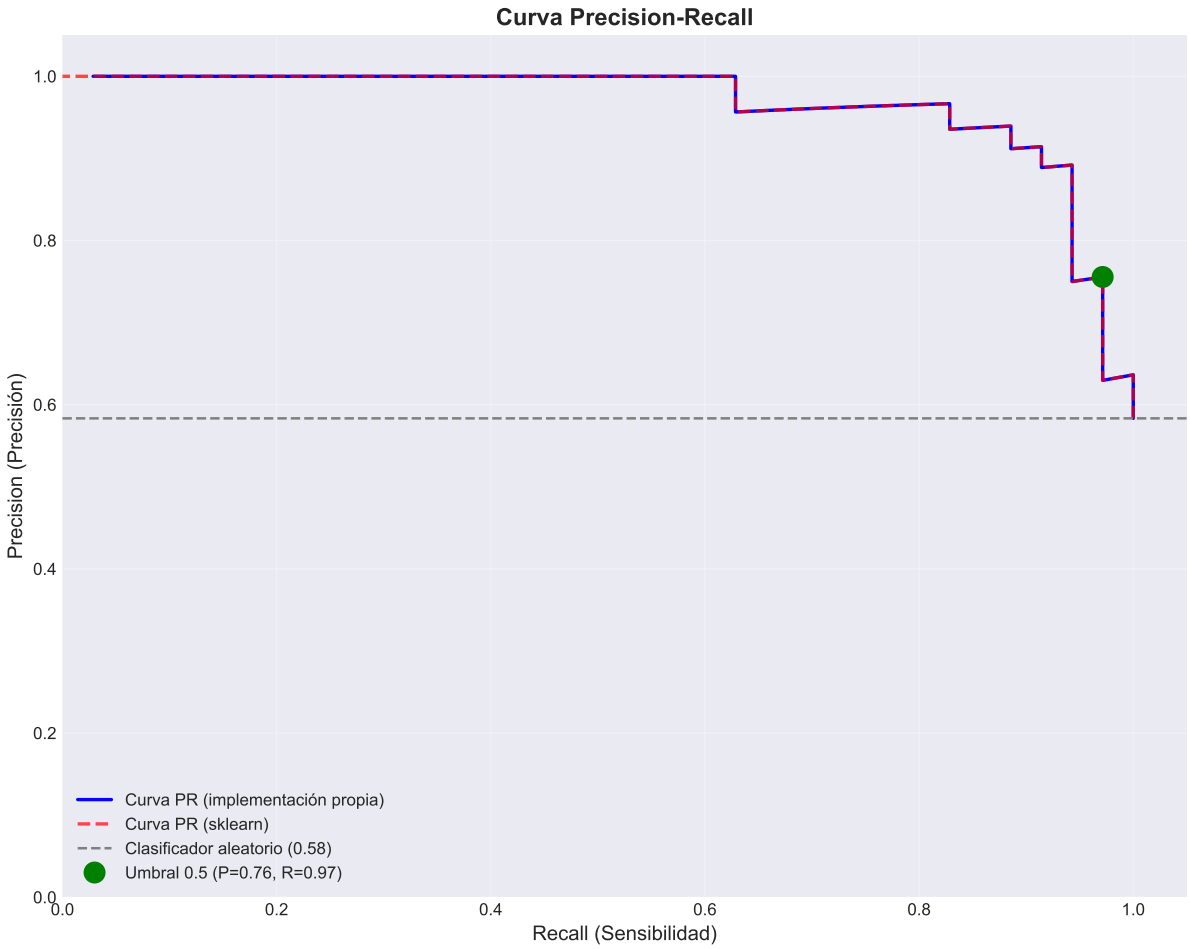
# Punto del modelo actual (umbral 0.5)
ax.plot(recall, precision, 'go', markersize=12,
        label=f'Umbral 0.5 (P={precision:.2f}, R={recall:.2f})')

ax.set_xlabel('Recall (Sensibilidad)', fontsize=12)
ax.set_ylabel('Precision (Precisión)', fontsize=12)
ax.set_title('Curva Precision-Recall', fontsize=14, fontweight='bold')
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
ax.set_xlim([0, 1.05])
ax.set_ylim([0, 1.05])

plt.tight_layout()
plt.savefig('curva_precision_recall.png', dpi=300, bbox_inches='tight')
```

```
print("Curva Precision-Recall guardada como 'curva_precision_recall.png'")
```

```
Curva Precision-Recall guardada como 'curva_precision_recall.png'
```



11 Curva ROC y AUC

11.1 Concepto de ROC

```
print("""
CURVA ROC (Receiver Operating Characteristic)

La curva ROC grafica:
- Eje X: False Positive Rate (FPR) = FP / (FP + TN)
- Eje Y: True Positive Rate (TPR) = TP / (TP + FN)  [mismo que Recall]

Interpretación:
- Diagonal (línea punteada): Clasificador aleatorio (AUC = 0.5)
- Curva sobre diagonal: Mejor que aleatorio
- Más cerca de esquina superior izquierda: Mejor modelo
- AUC (Area Under Curve): Métrica resumen
  * AUC = 1.0: Clasificador perfecto
```

```
  * AUC = 0.5: Clasificador aleatorio
  * AUC < 0.5: Peor que aleatorio
""")
```

```
CURVA ROC (Receiver Operating Characteristic)

La curva ROC grafica:
- Eje X: False Positive Rate (FPR) = FP / (FP + TN)
- Eje Y: True Positive Rate (TPR) = TP / (TP + FN)  [mismo que Recall]

Interpretación:
- Diagonal (línea punteada): Clasificador aleatorio (AUC = 0.5)
- Curva sobre diagonal: Mejor que aleatorio
- Más cerca de esquina superior izquierda: Mejor modelo
- AUC (Area Under Curve): Métrica resumen
  * AUC = 1.0: Clasificador perfecto
  * AUC = 0.5: Clasificador aleatorio
  * AUC < 0.5: Peor que aleatorio
```

11.2 Calcular TPR y FPR

```
def calcular_tpr(y_real, y_pred):
    """Calcula True Positive Rate (TPR) = Recall"""
    return calcular_recall(y_real, y_pred)

def calcular_fpr(y_real, y_pred):
    """Calcula False Positive Rate (FPR)"""
    _, TP, TN, FP, FN = calcular_matriz_confusion(y_real, y_pred)

    if FP + TN == 0:
        return 0.0

    fpr = FP / (FP + TN)
    return fpr

def calcular_curva_roc(y_real, probabilidades):
    """
    Calcula curva ROC para diferentes umbrales
    """
    umbrales = np.sort(np.unique(probabilidades))

    tpr_list = []
    fpr_list = []

    for umbral in umbrales:
        y_pred = (probabilidades >= umbral).astype(int)
```

```
        tpr = calcular_tpr(y_real, y_pred)
        fpr = calcular_fpr(y_real, y_pred)

        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return np.array(fpr_list), np.array(tpr_list), umbrales

# Calcular curva ROC
fpr_curva, tpr_curva, umbrales_roc = calcular_curva_roc(
    y_test, probabilidades[:, 1]
)

# También usar sklearn
fpr_sklearn, tpr_sklearn, _ = roc_curve(y_test, probabilidades[:, 1])

# Calcular AUC
roc_auc = auc(fpr_sklearn, tpr_sklearn)

print(f"AUC (Area Under Curve): {roc_auc:.4f}")
```

```
AUC (Area Under Curve): 0.9440
```

11.3 Visualizar curva ROC

```
# Crear gráfico
fig, ax = plt.subplots(figsize=(10, 8))

# Curva ROC
ax.plot(fpr_sklearn, tpr_sklearn, 'b-', linewidth=2,
        label=f'Curva ROC (AUC = {roc_auc:.3f})')

# Línea diagonal (clasificador aleatorio)
ax.plot([0, 1], [0, 1], 'r--', linewidth=2, label='Clasificador aleatorio (AUC = 0.5)

# Punto del modelo actual
fpr_actual = calcular_fpr(y_test, y_pred)
tpr_actual = calcular_tpr(y_test, y_pred)
ax.plot(fpr_actual, tpr_actual, 'go', markersize=12,
        label=f'Umbral 0.5 (FPR={fpr_actual:.2f}, TPR={tpr_actual:.2f})')

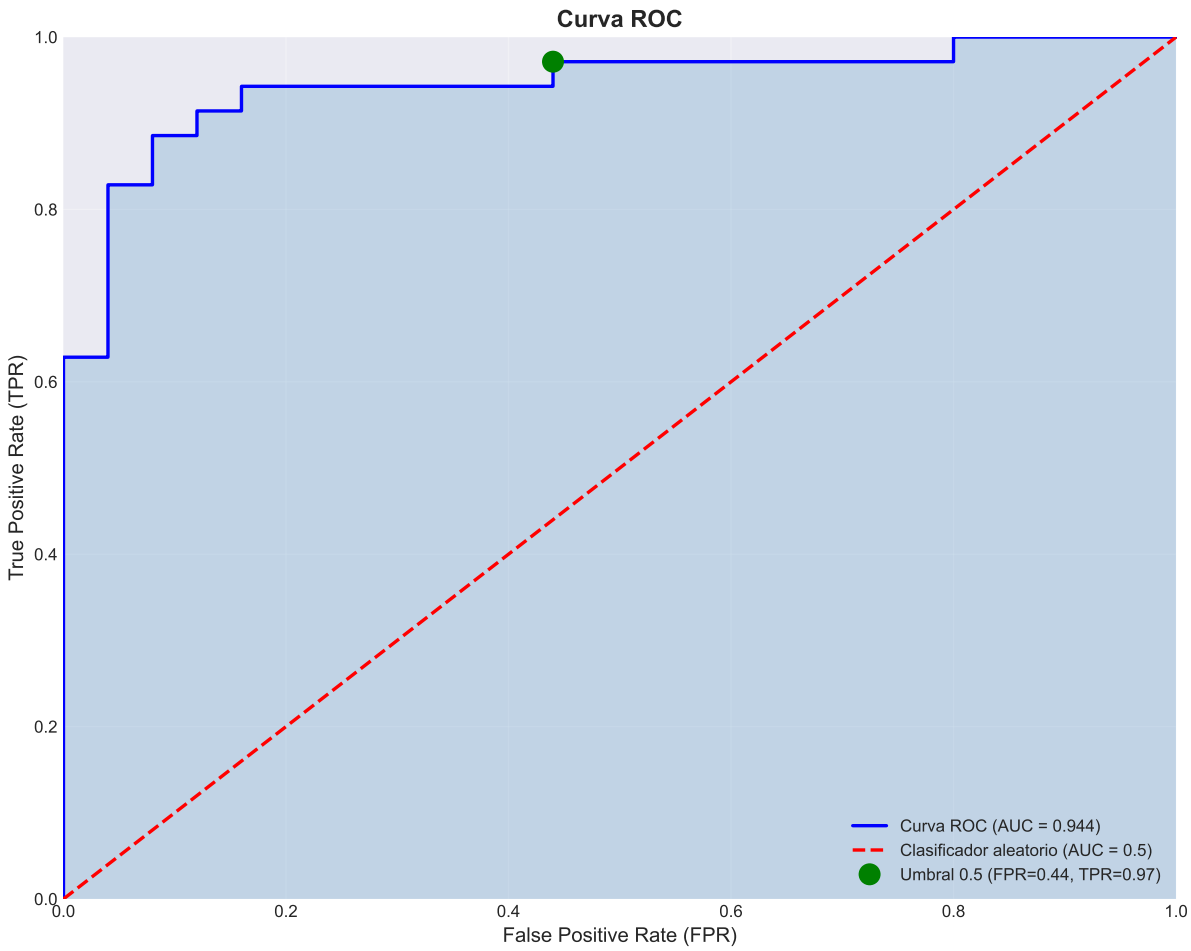
ax.set_xlabel('False Positive Rate (FPR)', fontsize=12)
ax.set_ylabel('True Positive Rate (TPR)', fontsize=12)
ax.set_title('Curva ROC', fontsize=14, fontweight='bold')
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
ax.set_xlim([0, 1])
```

```
ax.set_ylim([0, 1])

# Área sombreada bajo la curva
ax.fill_between(fpr_sklearn, tpr_sklearn, alpha=0.2, label='AUC')

plt.tight_layout()
plt.savefig('curva_roc.png', dpi=300, bbox_inches='tight')
print("Curva ROC guardada como 'curva_roc.png'")
```

```
Curva ROC guardada como 'curva_roc.png'
```



11.4 Interpretación del AUC

```
def interpretar_auc(auc_score):
    """Interpreta el valor de AUC"""
    if auc_score >= 0.9:
        return "Excelente"
    elif auc_score >= 0.8:
        return "Muy bueno"
    elif auc_score >= 0.7:
        return "Bueno"
```

```
    elif auc_score >= 0.6:
        return "Regular"
    else:
        return "Pobre"

interpretacion = interpretar_auc(roc_auc)

print(f"\n\nINTERPRETACIÓN DEL MODELO")
print("=" * 70)
print(f"AUC: {roc_auc:.4f} - {interpretacion}")
print(f"\nSignificado:")
print(f"Si tomamos un caso positivo y uno negativo al azar,")
print(f"el modelo asignará una probabilidad mayor al caso positivo")
print(f"el {roc_auc*100:.1f}% de las veces.")
```

```
INTERPRETACIÓN DEL MODELO
======================================================================
AUC: 0.9440 - Excelente

Significado:
Si tomamos un caso positivo y uno negativo al azar,
el modelo asignará una probabilidad mayor al caso positivo
el 94.4% de las veces.
```

12 Aplicaciones económicas

12.1 Aplicación 1: Predicción de incumplimiento crediticio

```
print("\n\nAPLICACIÓN: PREDICCIÓN DE INCUMPLIMIENTO CREDITICIO")
print("=" * 70)

# Generar datos simulados
np.random.seed(42)
n_clientes = 1000

datos_credito = pd.DataFrame({
    'edad': np.random.randint(18, 70, n_clientes),
    'ingreso_mensual': np.random.uniform(1000, 10000, n_clientes),
    'monto_credito': np.random.uniform(5000, 100000, n_clientes),
    'historial_pagos': np.random.randint(1, 10, n_clientes),
    'num_creditos_activos': np.random.randint(0, 5, n_clientes),
    'ratio_deuda_ingreso': np.random.uniform(0.1, 0.8, n_clientes)
})

# Variable objetivo: incumplimiento (1 = sí, 0 = no)
score_riesgo = (
```

```
    -datos_credito['ingreso_mensual'] * 0.0001 +
    datos_credito['monto_credito'] * 0.00001 +
    datos_credito['ratio_deuda_ingreso'] * 2 +
    -datos_credito['historial_pagos'] * 0.15 +
    datos_credito['num_creditos_activos'] * 0.2 +
    np.random.normal(0, 0.3, n_clientes)
)

datos_credito['incumplimiento'] = (score_riesgo > 0.5).astype(int)

print(f"\nDataset de créditos:")
print(f"  Total clientes: {n_clientes}")
print(f"  Incumplimientos: {datos_credito['incumplimiento'].sum()} ({datos_credito['i

# Dividir datos
X_credito = datos_credito.drop('incumplimiento', axis=1)
y_credito = datos_credito['incumplimiento']

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_credito, y_credito, test_size=0.3, random_state=42
)

# Entrenar modelo
modelo_credito = LogisticRegression(random_state=42, max_iter=1000)
modelo_credito.fit(X_train_c, y_train_c)

# Predecir
y_pred_c = modelo_credito.predict(X_test_c)
prob_incumplimiento = modelo_credito.predict_proba(X_test_c)

# Calcular métricas
cm_credito = confusion_matrix(y_test_c, y_pred_c)
_, TP_c, TN_c, FP_c, FN_c = calcular_matriz_confusion(y_test_c, y_pred_c)

accuracy_c = calcular_accuracy(y_test_c, y_pred_c)
precision_c = calcular_precision(y_test_c, y_pred_c)
recall_c = calcular_recall(y_test_c, y_pred_c)
f1_c = calcular_f1(y_test_c, y_pred_c)

# Curva ROC
fpr_c, tpr_c, _ = roc_curve(y_test_c, prob_incumplimiento[:, 1])
auc_c = auc(fpr_c, tpr_c)

print(f"\n\nRESULTADOS DEL MODELO")
print("=" * 70)
print(f"\nMatriz de Confusión:")
print(f"                    Predicción")
```

```
print(f"              Pagará  Incumplirá")
print(f"Real  Pagará    {TN_c:4d}      {FP_c:4d}")
print(f"   Incumplirá   {FN_c:4d}      {TP_c:4d}")

print(f"\n\nMétricas:")
print(f"  Accuracy:  {accuracy_c:.4f} ({accuracy_c*100:.1f}%)")
print(f"  Precision: {precision_c:.4f} ({precision_c*100:.1f}%)")
print(f"  Recall:    {recall_c:.4f} ({recall_c*100:.1f}%)")
print(f"  F1-Score:  {f1_c:.4f}")
print(f"  AUC:       {auc_c:.4f}")

# Análisis de costos
monto_promedio = datos_credito['monto_credito'].mean()
tasa_recuperacion = 0.30  # Se recupera 30% en promedio

costo_fn = FN_c * monto_promedio * (1 - tasa_recuperacion)  # No detectar incumplimie
costo_fp = FP_c * 500  # Costo de revisar caso que no es incumplimiento

print(f"\n\nANÁLISIS DE COSTOS")
print("=" * 70)
print(f"  Monto promedio por crédito: S/ {monto_promedio:,.2f}")
print(f"\n  Falsos Negativos (FN): {FN_c}")
print(f"    → Incumplimientos no detectados")
print(f"    → Pérdida estimada: S/ {costo_fn:,.2f}")
print(f"\n  Falsos Positivos (FP): {FP_c}")
print(f"    → Clientes buenos rechazados o revisados")
print(f"    → Costo de revisión: S/ {costo_fp:,.2f}")
print(f"\n  Costo total: S/ {costo_fn + costo_fp:,.2f}")

print(f"\n\nRECOMENDACIÓN:")
if recall_c < 0.7:
    print("  RECALL BAJO: Estamos perdiendo muchos incumplimientos.")
    print("  Considerar reducir umbral para detectar más casos de riesgo.")
if precision_c < 0.5:
    print("  PRECISION BAJA: Muchas falsas alarmas.")
    print("  Considerar aumentar umbral o mejorar features.")
```

```
APLICACIÓN: PREDICCIÓN DE INCUMPLIMIENTO CREDITICIO
======================================================================

Dataset de créditos:
  Total clientes: 1000
  Incumplimientos: 500 (50.0%)


RESULTADOS DEL MODELO
======================================================================
```

```
Matriz de Confusión:
                    Predicción
              Pagará  Incumplirá
Real  Pagará     124        19
   Incumplirá     22       135


Métricas:
  Accuracy:  0.8633 (86.3%)
  Precision: 0.8766 (87.7%)
  Recall:    0.8599 (86.0%)
  F1-Score:  0.8682
  AUC:       0.9521


ANÁLISIS DE COSTOS
======================================================================
  Monto promedio por crédito: S/ 52,452.51

  Falsos Negativos (FN): 22
    → Incumplimientos no detectados
    → Pérdida estimada: S/ 807,768.67

  Falsos Positivos (FP): 19
    → Clientes buenos rechazados o revisados
    → Costo de revisión: S/ 9,500.00

  Costo total: S/ 817,268.67


RECOMENDACIÓN:
```

12.2 Aplicación 2: Clasificación de empresas por riesgo de quiebra

```
print("\n\n\nAPLICACIÓN: CLASIFICACIÓN DE RIESGO DE QUIEBRA EMPRESARIAL")
print("=" * 70)

# Generar datos de empresas
np.random.seed(42)
n_empresas = 500

datos_empresas = pd.DataFrame({
    'activos_totales': np.random.uniform(100000, 10000000, n_empresas),
    'pasivos_totales': np.random.uniform(50000, 8000000, n_empresas),
    'ventas_anuales': np.random.uniform(200000, 15000000, n_empresas),
    'utilidad_neta': np.random.uniform(-500000, 2000000, n_empresas),
    'flujo_caja': np.random.uniform(-200000, 1000000, n_empresas),
```

```
    'años_operacion': np.random.randint(1, 30, n_empresas)
})

# Calcular ratios financieros
datos_empresas['ratio_liquidez'] = datos_empresas['activos_totales'] / datos_empresas
datos_empresas['margen_utilidad'] = datos_empresas['utilidad_neta'] / datos_empresas[
datos_empresas['ratio_endeudamiento'] = datos_empresas['pasivos_totales'] / datos_emp

# Variable objetivo: quiebra (1 = sí, 0 = no)
score_quiebra = (
    -datos_empresas['ratio_liquidez'] * 0.5 +
    -datos_empresas['margen_utilidad'] * 2 +
    datos_empresas['ratio_endeudamiento'] * 3 +
    -datos_empresas['flujo_caja'] * 0.000001 +
    -datos_empresas['años_operacion'] * 0.05 +
    np.random.normal(0, 0.5, n_empresas)
)

datos_empresas['quiebra'] = (score_quiebra > 1.0).astype(int)

print(f"\nDataset de empresas:")
print(f"  Total empresas: {n_empresas}")
print(f"  Quiebras: {datos_empresas['quiebra'].sum()} ({datos_empresas['quiebra'].mea

# Seleccionar features
features_empresas = ['ratio_liquidez', 'margen_utilidad', 'ratio_endeudamiento',
                     'flujo_caja', 'años_operacion']

X_empresas = datos_empresas[features_empresas]
y_empresas = datos_empresas['quiebra']

# Dividir datos
X_train_e, X_test_e, y_train_e, y_test_e = train_test_split(
    X_empresas, y_empresas, test_size=0.3, random_state=42
)

# Entrenar modelo
modelo_quiebra = LogisticRegression(random_state=42, max_iter=1000)
modelo_quiebra.fit(X_train_e, y_train_e)

# Evaluar
y_pred_e = modelo_quiebra.predict(X_test_e)
prob_quiebra = modelo_quiebra.predict_proba(X_test_e)

# Métricas
accuracy_e = calcular_accuracy(y_test_e, y_pred_e)
precision_e = calcular_precision(y_test_e, y_pred_e)
```

```
recall_e = calcular_recall(y_test_e, y_pred_e)
f1_e = calcular_f1(y_test_e, y_pred_e)

fpr_e, tpr_e, _ = roc_curve(y_test_e, prob_quiebra[:, 1])
auc_e = auc(fpr_e, tpr_e)

print(f"\n\nMÉTRICAS DEL MODELO")
print("=" * 70)
print(f"  Accuracy:  {accuracy_e:.4f}")
print(f"  Precision: {precision_e:.4f}")
print(f"  Recall:    {recall_e:.4f}")
print(f"  F1-Score:  {f1_e:.4f}")
print(f"  AUC:       {auc_e:.4f}")

# Importancia de variables
print(f"\n\nIMPORTANCIA DE VARIABLES (Coeficientes)")
print("=" * 70)
coeficientes = pd.DataFrame({
    'Variable': features_empresas,
    'Coeficiente': modelo_quiebra.coef_[0]
}).sort_values('Coeficiente', key=abs, ascending=False)

for _, row in coeficientes.iterrows():
    signo = "↑ Mayor riesgo" if row['Coeficiente'] > 0 else "↓ Menor riesgo"
    print(f"  {row['Variable']:<25} {row['Coeficiente']:>8.4f}  {signo}")

print(f"\n\nAPLICACIÓN PRÁCTICA:")
print("Este modelo puede usarse para:")
print("  1. Sistema de alerta temprana de quiebra")
print("  2. Decisiones de inversión o préstamos corporativos")
print("  3. Priorización de auditorías o intervenciones")
print("  4. Análisis de cartera de inversiones")
```

```
APLICACIÓN: CLASIFICACIÓN DE RIESGO DE QUIEBRA EMPRESARIAL
======================================================================

Dataset de empresas:
  Total empresas: 500
  Quiebras: 197 (39.4%)


MÉTRICAS DEL MODELO
======================================================================
  Accuracy:  0.9333
  Precision: 0.9048
  Recall:    0.9344
```

```
  F1-Score:  0.9194
  AUC:       0.9854


IMPORTANCIA DE VARIABLES (Coeficientes)
======================================================================
  ratio_liquidez             -3.2044  ↓ Menor riesgo
  ratio_endeudamiento         2.6668  ↑ Mayor riesgo
  margen_utilidad            -2.3090  ↓ Menor riesgo
  años_operacion             -0.0526  ↓ Menor riesgo
  flujo_caja                 -0.0000  ↓ Menor riesgo


APLICACIÓN PRÁCTICA:
Este modelo puede usarse para:
  1. Sistema de alerta temprana de quiebra
  2. Decisiones de inversión o préstamos corporativos
  3. Priorización de auditorías o intervenciones
  4. Análisis de cartera de inversiones
```

13 Ejercicios prácticos

13.1 Ejercicio 1: Optimizar umbral según costo

```python
print("\n\n\nEJERCICIO: OPTIMIZACIÓN DE UMBRAL SEGÚN COSTOS")
print("=" * 70)

def encontrar_umbral_optimo(y_real, probabilidades, costo_fp, costo_fn):
    """
    Encuentra el umbral óptimo que minimiza costos totales
    """
    umbrales = np.linspace(0.1, 0.9, 50)

    mejores_resultados = {
        'umbral': None,
        'costo_total': float('inf'),
        'precision': 0,
        'recall': 0,
        'fp': 0,
        'fn': 0
    }

    resultados = []

    for umbral in umbrales:
        y_pred = (probabilidades >= umbral).astype(int)

        _, TP, TN, FP, FN = calcular_matriz_confusion(y_real, y_pred)
```

```
        costo_total = FP * costo_fp + FN * costo_fn

        prec = calcular_precision(y_real, y_pred)
        rec = calcular_recall(y_real, y_pred)

        resultados.append({
            'umbral': umbral,
            'costo_total': costo_total,
            'precision': prec,
            'recall': rec,
            'fp': FP,
            'fn': FN
        })

        if costo_total < mejores_resultados['costo_total']:
            mejores_resultados = {
                'umbral': umbral,
                'costo_total': costo_total,
                'precision': prec,
                'recall': rec,
                'fp': FP,
                'fn': FN
            }

    return mejores_resultados, pd.DataFrame(resultados)

# Aplicar al modelo de crédito
costo_por_fp = 500  # Costo de revisar un caso falso positivo
costo_por_fn = monto_promedio * 0.7  # 70% del monto no recuperado

resultado_optimo, df_resultados = encontrar_umbral_optimo(
    y_test_c,
    prob_incumplimiento[:, 1],
    costo_por_fp,
    costo_por_fn
)

print(f"\nCostos considerados:")
print(f"  Costo por Falso Positivo: S/ {costo_por_fp:,.2f}")
print(f"  Costo por Falso Negativo: S/ {costo_por_fn:,.2f}")

print(f"\n\nUMBRAL ÓPTIMO ENCONTRADO:")
print("=" * 70)
print(f"  Umbral óptimo: {resultado_optimo['umbral']:.3f}")
print(f"  Costo total: S/ {resultado_optimo['costo_total']:,.2f}")
print(f"  Precision: {resultado_optimo['precision']:.4f}")
print(f"  Recall: {resultado_optimo['recall']:.4f}")
```

```
print(f"  Falsos Positivos: {resultado_optimo['fp']}")
print(f"  Falsos Negativos: {resultado_optimo['fn']}")

# Comparar con umbral 0.5
_, TP_05, TN_05, FP_05, FN_05 = calcular_matriz_confusion(y_test_c, y_pred_c)
costo_umbral_05 = FP_05 * costo_por_fp + FN_05 * costo_por_fn

print(f"\n\nCOMPARACIÓN CON UMBRAL 0.5:")
print(f"  Costo con umbral 0.5: S/ {costo_umbral_05:,.2f}")
print(f"  Ahorro con umbral óptimo: S/ {costo_umbral_05 - resultado_optimo['costo_tot

# Graficar
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Gráfico de costos
ax1.plot(df_resultados['umbral'], df_resultados['costo_total'], 'b-', linewidth=2)
ax1.axvline(x=resultado_optimo['umbral'], color='r', linestyle='--',
            label=f"Umbral óptimo ({resultado_optimo['umbral']:.3f})")
ax1.axvline(x=0.5, color='g', linestyle='--', alpha=0.5, label='Umbral 0.5')
ax1.set_xlabel('Umbral')
ax1.set_ylabel('Costo Total (S/)')
ax1.set_title('Costo Total vs Umbral')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Gráfico de Precision vs Recall
ax2.plot(df_resultados['umbral'], df_resultados['precision'], 'b-',
         linewidth=2, label='Precision')
ax2.plot(df_resultados['umbral'], df_resultados['recall'], 'r-',
         linewidth=2, label='Recall')
ax2.axvline(x=resultado_optimo['umbral'], color='g', linestyle='--', alpha=0.5,
            label='Umbral óptimo')
ax2.set_xlabel('Umbral')
ax2.set_ylabel('Métrica')
ax2.set_title('Precision y Recall vs Umbral')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('optimizacion_umbral.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'optimizacion_umbral.png'")
```

```
EJERCICIO: OPTIMIZACIÓN DE UMBRAL SEGÚN COSTOS
======================================================================

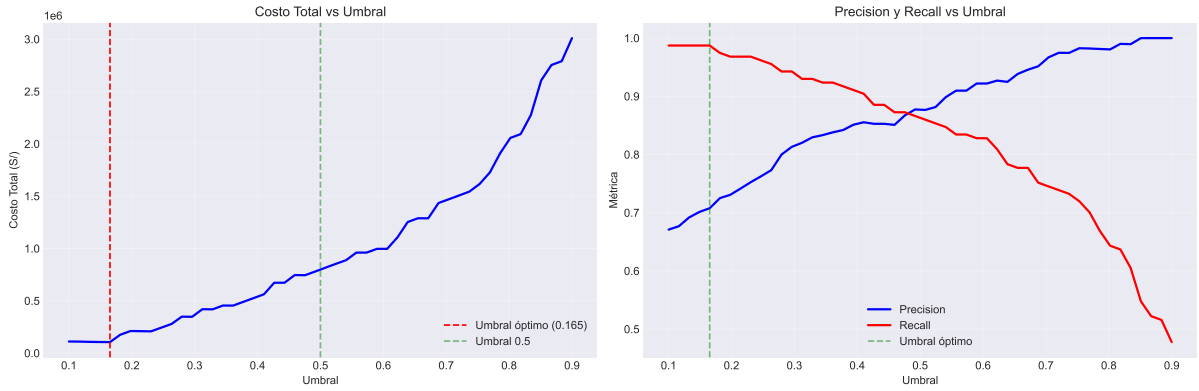
Costos considerados:
```

```
  Costo por Falso Positivo: S/ 500.00
  Costo por Falso Negativo: S/ 36,716.76


UMBRAL ÓPTIMO ENCONTRADO:
======================================================================
  Umbral óptimo: 0.165
  Costo total: S/ 105,433.52
  Precision: 0.7078
  Recall: 0.9873
  Falsos Positivos: 64
  Falsos Negativos: 2


COMPARACIÓN CON UMBRAL 0.5:
  Costo con umbral 0.5: S/ 817,268.67
  Ahorro con umbral óptimo: S/ 711,835.15

Gráfico guardado como 'optimizacion_umbral.png'
```



14 Conclusión

En esta guía hemos explorado predicción y métricas de performance:

El problema de predicción

Predecir eventos futuros basándose en datos históricos para tomar mejores decisiones.

Matriz de confusión

Herramienta fundamental que muestra TP, TN, FP, FN para evaluar clasificadores.

Métricas de clasificación

- Accuracy: Proporción de predicciones correctas
- Precision: De las predicciones positivas, cuántas son correctas
- Recall: De los casos positivos reales, cuántos detectamos
- F1-Score: Balance entre precision y recall

Curvas de evaluación

- Curva Precision-Recall: Para evaluar trade-off entre precision y recall
- Curva ROC: Para evaluar capacidad discriminativa del modelo
- AUC: Métrica resumen de la curva ROC

15 Próximos pasos

En la siguiente guía (Guía 9: Métodos de ML para Clasificación) exploraremos:

- Regresión logística
- K-Nearest Neighbors (KNN)
- Árboles de decisión
- Random Forests
- Clasificación multi-clase

Estos algoritmos nos permitirán construir modelos predictivos más sofisticados para problemas económicos complejos.

16 Recursos adicionales

Para profundizar en métricas de clasificación:

- Scikit-learn Metrics: scikit-learn.org/stable/modules/model_evaluation.html
- Understanding ROC Curves: developers.google.com/machine-learning/crash-course/classification/roc-and-auc
- Precision-Recall trade-off: machinelearningmastery.com
- Aplicaciones en economía: papers en credit scoring, fraud detection

17 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!